

操作系统第八周星期一讲稿

💡 课前提醒：这学期实验比较多，也比较难，大家要提前做。尤其是Live系列，代码量不大，但核心的那几行很重要，一定要看明白。我们也尽量在题目上给大家提示重点看哪些代码。

一、考研那些事儿

先跟大家说几句，最近我参加了一些研究生面试，有一些感受。大家都知道，北航现在计算机考研用的是408，四门课：操作系统、数据结构、组成原理、计算机网络。面试的时候我问了一些同学，基本概念还行，但一深入到实现层面，很多就答不上来了。

比如我问：

- 虚拟地址和物理地址的区别是什么？
- 页表是谁填的？
- 访问一个受保护的地址会发生什么？

很多就答不上来了。这就是为什么我们这学期加了这么多实验，就是希望大家不仅知道书本上的原理，还要知道它背后是怎么实现的。

🔔 敲黑板：知其然，还要知其所以然。只有知道怎么实现的，再反过来背原理的时候，才更容易记忆。

二、复习：进程同步与互斥的基本思路

上节课我们讲了要实现进程同步与互斥，技术手段有哪些：

1. 纯软件方法

完全用代码实现互斥，比如Peterson算法、Dekker算法。

- 优点：不需要硬件支持
- 缺点：写起来比较复杂，而且是**忙等**，CPU空转浪费

2. 硬件辅助方法

用Test-and-Set或者Exchange指令，原子性地做两件事。

- 优点：实现简单，几行代码搞定
- 缺点：还是**忙等**

3. 问题：盲等太浪费CPU

大家想想，我去银行排队，如果队很长，我非要站在那排队，这不就是忙等吗？我站在那什么也干不了，时间都浪费了。

能不能改成：我取个号，回家等着，到号了银行给我打电话叫我？

这就是我们要引入信号量的初衷——把盲等变成阻塞式等待。

三、信号量机制详解

3.1 信号量是什么

信号量 (Semaphore)，你可以理解成封装了一个特殊变量，还有两个原子操作：

操作	含义	作用
P	Passeren (荷兰语"尝试")	申请资源，可能阻塞
V	Verhogen (荷兰语"增加")	释放资源，可能唤醒

三大原则：

1. 信号量必须设初值，而且只能设一次
2. 只能通过P、V操作修改信号量的值
3. P、V操作本身是原子的，执行中间不能被打断

💡 为什么P、V要用原子操作？你想想，如果两个进程同时做P操作，如果中间能被打断，那信号量的值不就乱了吗？

3.2 信号量的物理意义 ★★ ★ (考试重点)

这一页考试要考！给我记死了：

信号量S的值	物理含义
$S > 0$	表示还有S个可用资源
$S = 0$	资源刚好用完，还没有进程在等
$S < 0$	表示有

- **P操作** = 申请一个资源： $S = S - 1$
 - 如果减完还 ≥ 0 ：恭喜你，拿到资源了，继续执行
 - 如果减完 < 0 ：不好意思，资源不够了，你去排队睡觉吧（变成waiting状态）
- **V操作** = 释放一个资源： $S = S + 1$
 - 如果加完还 ≤ 0 ：说明还有人在排队等，那得从队列里唤醒一个进程
 - 如果加完 > 0 ：说明没人在等，那就完事儿了

⚠ 注意：这里的阻塞是真的把CPU让出来，进程进入waiting状态，不是空转！直到V操作把它唤醒，它才从waiting变成ready，等着重新调度。

3.3 信号量的实现细节

信号量本身的P、V操作要保证原子性，怎么实现？

- 可以用关中断

- 可以用前面说的TS指令（自旋锁）

有人会问：你这不还是用了忙等吗？

- 没错，P、V操作本身是用了一点点忙等
- 但P、V操作本身非常短，就几行代码
- 关键是：它避免了进程在**用户临界区外面**长时间忙等！

这就好比：你去图书馆占座，你不用一直在座位旁边站着等（长忙等），你可以去服务台登记一下（P操作，很短的忙等），然后去干别的，有座了图书馆给你打电话。

四、用信号量解决实际问题

我们整个这一章，其实就两大类问题：**互斥和同步**。

信号量都能搞定！

4.1 模式一：用信号量实现互斥

问题场景：临界区一次只能进一个进程。

解决方案：

- 设一个互斥信号量 mutex，初值 = 1
- 每个进程在进入临界区之前执行 P(mutex)
- 退出临界区之后执行 V(mutex)

进程A:

```
P(mutex)      // 申请进门资格
临界区代码    // 进去了
V(mutex)      // 出来了，释放资格
```

进程B:

```
P(mutex)      // 申请进门资格
临界区代码    // 进去了
V(mutex)      // 出来了
```

执行过程分析：

- 如果A先进去，mutex变成0
- B再来P(mutex)，就会阻塞在那
- 直到A执行V(mutex)，mutex变回1，B才被唤醒
- 完美实现“一次只能进一个”

💡 这个mutex，很多语言里直接叫Mutex（互斥锁），就是这么来的。本质就是个二元信号量，要么0要么1。

4.2 模式二：用信号量实现同步

问题场景：两个进程要协调步伐，规定好了某段代码必须在另一段代码之后执行。

- 比如：你必须先做完作业，才能出去玩

解决方案：

- 设一个同步信号量 S ，初值 = 0
- **后执行**的代码前面加 $P(S)$
- **先执行**的代码后面加 $V(S)$

```
进程A（先执行）：
    代码段A
    V(S)          // 我做完了，发个信号

进程B（后执行）：
    P(S)          // 等等信号再说
    代码段B
```

为什么初值是0？

- 一开始 $S=0$ ，如果B先执行到 $P(S)$ ，就会阻塞，等A做完
- 不管两个进程谁快谁慢，最终一定是A的代码在B前面执行

这就实现了强制性的先后顺序！

4.3 推广：复杂前驱图的实现

刚才才是两个进程的先后关系，那如果更复杂呢？比如：

- A和B都做完了，C才能开始
- C做完了，D才能开始

```
A ----\
      --> C --> D
B ----/
```

实现方法：

- 设三个信号量： S_a 、 S_b 、 S_c ，初值都是0
- A做完后执行 $V(S_a)$
- B做完后执行 $V(S_b)$
- C开始前执行 $P(S_a)$ ； $P(S_b)$
- C做完后执行 $V(S_c)$
- D开始前执行 $P(S_c)$

完美！不管多复杂的前驱图，你只要给每条边设一个初值为0的信号量，该加P的地方加P，该加V的地方加V，就搞定了。

五、信号量经典应用模式总结

应用模式	信号量初值	使用场景
互斥	1	临界区，一次只能一个进
有限并发	C (C>1)	限制最多C个进程同时访问某个资源 比如教室限选80人，信号量设80，第81个就进不来
同步	0	协调先后顺序，前一个做完了后一个才能开始

就这三板斧！大部分同步互斥问题，都是这三种模式的组合。

六、进阶问题：屏障（Barrier）语义

6.1 什么是屏障需求？

刚才说的同步是“你先做，做完了我再做”，那如果我要：

等所有人都到齐了，再一起出发

比如：

- 大家约好一起去博物馆，人不齐不发车
- 分布式训练神经网络，所有机器这轮都算完了，再汇总结果
- 长跑比赛，所有人都冲过终点线了，再颁奖

这种需求，就叫**屏障（Barrier）**，也叫“汇合点”。

屏障的核心语义：

在所有进程都到达屏障点之前，先到的进程必须在屏障点等待，不能先走。

这既不是互斥，也不是简单的同步，这是一种更复杂的协调需求。

6.2 屏障怎么实现？

用我们前面学的信号量，就能实现！

我们需要三样东西：

1. **count**：普通变量，记录已经到达屏障点的进程数，初值=0
2. **mutex**：互斥信号量，初值=1，保护count的修改不冲突
3. **barrier**：同步信号量，初值=0，用来挡着大家等齐了再走

每个进程到达屏障点时，都执行这六行代码：

```
P(mutex);           // 要改count了，先上锁
count = count + 1;   // 我到了，报个到
V(mutex);           // 改完了，解锁

if (count == N) {    // 我是最后一个到的吗？
    V(barrier);       // 是的！我来开门，放第一个人过去
}
```

```
P(barrier);          // 到了先等着，门开了才能过
V(barrier);          // 我过去了，顺手把下一个人也放过去
```

6.3 为什么能工作？链式唤醒！

我们来走一遍流程（假设N=5个人）：

1. 前4个人陆续到达，count变成1、2、3、4
 - 都不满足count==N，所以都卡在P(barrier)那里睡觉
2. 第5个人到达，count变成5，满足条件了
 - 他执行V(barrier)，唤醒第一个人
3. 第一个人被唤醒，P(barrier)通过
 - 他立刻执行V(barrier)，又唤醒第二个人
4. 第二个人被唤醒，执行V(barrier)，唤醒第三个人
5. ...以此类推...

 这就是**链式唤醒**！像多米诺骨牌一样，一个推一个，最后所有人都通过了屏障。

而且最重要的是：**在最后一个人到达之前，没有任何人能通过屏障**！完美符合我们的需求。

本讲总结

今天讲的核心内容：

1. **信号量的本质**：从盲等变成阻塞等待，提高CPU利用率
2. **信号量物理意义**：正的是可用资源数，负的绝对值是等待进程数
3. **三大经典模式**：
 - 互斥：初值=1，临界区前后P、V
 - 有限并发：初值=C，限制最多C个同时访问
 - 同步：初值=0，先做的后面V，后做的前面P
4. **屏障实现**：用计数+互斥+链式唤醒，实现人齐了再一起走

信号量是操作系统并发控制的基石，下节课我们会用它来解决几个经典的“名题”，比如生产者-消费者问题、读者-写者问题、哲学家吃饭问题。

 **思考题**：你能想到生活中还有哪些屏障的例子？试着用刚才的六行代码实现一下。